Why Do You Need a Break?

One pattern for a whole family of sentences: ELIZA's "I am X" rule becomes a regular expression – and finally tells "mother" from "smothered".

Marcus Birkenkrahe

COSC 1210 · CS I · Augustana University

Fall 2026 · Downey, *Think Python* 3e, ch. 8

Review: mini challenge (10 min)

→ Switch to editor · mini challenge · 10 min, then we discuss

```
def normalize(sentence):
    sentence = sentence.lower()
    sentence = sentence.replace("i'm", "i am")
    return sentence.strip(".!?")

def i_am_reply(sentence):
    sentence = normalize(sentence)
    if sentence.startswith("i am "):
        return f"Why are you {sentence[5:]}?"
    return None
```

1. Write `you_are_reply(sentence)`: for a sentence that starts with "You are", return "What makes you think I am X?", where X is the rest of the sentence. For any other sentence, return `None`.

```
>>> you_are_reply("You are kind.")
'What makes you think I am kind?'
>>> you_are_reply("You are so clever!")
'What makes you think I am so clever?'
>>> print(you_are_reply("Are you a robot?"))
None
```

2. Change the check to `startswith("you are")` – without the space – and keep the slice. What does it return for `"You aren't boring."`, and why?

Review: mini challenge – solution

1. "you are " has 8 characters

```
def you_are_reply(sentence):
    sentence = normalize(sentence)
    if sentence.startswith("you are "):
        rest = sentence[8:]
        return f"What makes you think I am {rest}?"
    return None
```

```
What makes you think I am kind?
What makes you think I am so clever?
None
```

2. Without the space

```
def you_are_reply_nospace(sentence):
    sentence = normalize(sentence)
    if sentence.startswith("you are"):
        rest = sentence[8:]
        return f"What makes you think I am {rest}?"
    return None

print(you_are_reply_nospace(
    "You aren't boring."))
```

```
What makes you think I am 't boring?
```

Review: mini challenge – discussion

- `"you are "` has 8 characters with the space, so the rest starts at index 8. `len("you are ")` saves counting.
- Without the space, "you aren't" passes the check, and the slice cuts the word in the middle. The space in the check is a word boundary made by hand.
- `"You're kind."` returns `None`: `normalize` only writes out "i'm". A third `replace`? A regular expression handles all of them at once.

Review: catch the bug with a doctest (5 min)

→ Switch to editor · 5 min · both versions from the mini challenge

```
from doctest import run_docstring_examples

def run_doctests(func):
    run_docstring_examples(func, globals(),
                           name=func.__name__)
```

1. Three doctests in `you_are_reply`, for `"You are kind."`, `"Are you a robot?"` and `"You aren't boring."`. A string shows with quotes; `None` only with `print`
2. The same three in `you_are_reply_nospace`
3. Run `run_doctests` on both. Which one prints something?

`you_are_reply`: no output – all three pass.

`you_are_reply_nospace`:

```
File "__main__", line 24, in you_are_reply_nospace
Failed example:
    print(you_are_reply_nospace("You aren't boring."))
Expected:
    None
Got:
    What makes you think I am 't boring?
```

→ The first two examples pass for both versions. Only the edge case tells them apart.

Review: normalizing (5 min)

```python
def normalize(sentence):
    return (sentence.lower()
            .replace("i'm", "i am")
            .strip(".!?"))
```

```python
def i_am_reply(sentence):
    sentence = normalize(sentence)
    if sentence.startswith("i am"):
        return f"Why are you {sentence[5:]}?"
    return None
```

1. What does `normalize("I'M AWAKE!!!")` return?

→ `'i am awake'` – `strip(".!?")` removes all of these characters at both ends, not just one.

2. What does `i_am_reply("I am awake... really?")` return?

→ `'Why are you awake... really?'` – `strip` only works at the ends; the `...` in the middle stays.

3. With `.lower().replace("I'm", "I am")` instead: what does `i_am_reply("I'm awake.")` return?

→ `None` – after `lower()` there is no `"I'm"` left to replace. The order of the chain matters.

Search for a pattern

→ **Switch to editor · code-along 1**

```
import re

sentence = "I am awake and ready."
result = re.search("awake", sentence)
print(result)
print(result.group())
print(result.span())
```

```
<re.Match object; span=(5, 10), match='awake'>
awake
(5, 10)
```

- A **regular expression** is a pattern for strings. The module `re` searches with it
- Found: `re.search` returns a **match object**
- `group()`: the part that matched. `span()`: where it starts and ends, as in a slice

Not found – and found too often

Not found: `None`, like `find_keyword`.

```
print(re.search("happy", sentence))
```

```
None
```

→ Switch to editor · TRY (2 min) · predict, then run

```
print(re.search("am", "My hammer is new."))
```

```
<re.Match object; span=(4, 6), match='am'>
```

→ Same problem as `"no" in "know"`: a plain pattern also matches inside a word. Code-along 4 fixes it.

| means “or”

→ **Switch to editor · code-along 2**

```
family = "mother|father|sister|brother"
print(re.search(family, "My brother calls every day.").group())
print(re.search(family, "Our dog sleeps all day."))
```

```
brother
None
```

One pattern does the work of four `in` tests, one per family word. But:

```
print(re.search(family, "I smothered the pancakes in syrup.").group())
```

```
mother
```

^ and $: anchors

```
print(re.search("^i am", "i am happy"))
print(re.search("^i am", "you think i am smart"))
print(re.search("happy$", "i am happy"))
print(re.search("happy$", "happy about that"))
```

```
<re.Match object; span=(0, 4), match='i am'>
None
<re.Match object; span=(5, 10), match='happy'>
None
```

- ^ matches the start of the string, $ the end: they **anchor** the pattern
- "^i am" does what `startswith("i am")` did
- $ is the partner, like `endswith`

? optional, .* the rest, ( ) a group

→ **Switch to editor · code-along 3**

```
print(re.search("i'?m", "i'm awake").group())
print(re.search("i'?m", "im awake").group())
```

```
i'm
im
```

```
match = re.search("i am (.*)",
                  "i am awake and ready")
print(match.group())
print(match.group(1))
```

```
i am awake and ready
awake and ready
```

- ?: the character before it is optional
- . matches any one character, * repeats the one before: .* is "everything that follows"
- Parentheses make a **group**. group() is the whole match, group(1) the first group
- No more counting characters for sentence[5:]

“I am X” as a pattern

```
def normalize(sentence):
    sentence = sentence.lower()
    sentence = sentence.replace("i'm", "i am")
    return sentence.strip(".!?")

def i_am_reply(sentence):
    match = re.search("i am (.*)", normalize(sentence))
    if match is None:
        return None
    return f"Why are you {match.group(1)}?"

print(i_am_reply("I am awake and ready."))
print(i_am_reply("You are awake."))
```

```
Why are you awake and ready?
None
```

→ Switch to editor · TRY (2 min) · predict, then run

```
print(i_am_reply(
  "My mother says I am lucky."))
```

```
Why are you lucky?
```

→ The pattern matches anywhere – ELIZA calls *you* lucky.

Anchor the rule

```
def i_am_reply(sentence):
    match = re.search("^i am (.*)", normalize(sentence))
    if match is None:
        return None
    return f"Why are you {match.group(1)}?"

print(i_am_reply("My mother says I am lucky."))
print(i_am_reply("I am awake and ready."))
print(i_am_reply("Well, I am awake."))
```

```
None
Why are you awake and ready?
None
```

The price: “Well, I am awake.” gets no reply either.

“I am X” belongs at the start. Which patterns should match *anywhere*?

→ Keywords: “dream”, “always”, a family word – and “i need” in “Well, I need coffee”.

re.sub: replace with a pattern

→ **Switch to editor · code-along 4**

```
print(re.sub("i'?m", "i am", "im awake"))
print(re.sub("i'?m", "i am", "tell him i'm happy"))
```

```
i am awake
tell hi am i am happy
```

"him" contains "im". \b matches a **word boundary**: the edge between a letter and a space, a punctuation mark or the end. It matches no character, only the edge.

```
print(re.sub(r"\bi'?m\b", "i am", "tell him i'm happy"))
print(re.search(r"\bmother\b", "i smothered the pancakes in syrup"))
```

```
tell him i am happy
None
```

Raw strings

→ **Switch to editor · TRY (2 min) · leave out the r. What happens?**

```
print(re.search("\bmother\b",
                "my mother is busy"))
print(len("\b"), len(r"\b"))
```

```
None
1 2
```

Without `r`, Python turns `\b` into one character, a "backspace", before `re` sees it. The **raw string** `r"..."` keeps the backslash.

```
def normalize(sentence):
    sentence = sentence.lower()
    sentence = re.sub(r"\bi'?m\b", "i am",
                      sentence)
    return sentence.strip(".!?")

family = r"\b(mother|father|sister|brother)\b"
print(i_am_reply("Im awake."))
print(re.search(family,
               "I smothered the pancakes in syrup."))
```

```
Why are you awake?
None
```

Practice: why do you need a break?

→ **Switch to editor · 10 min, then 5 min discussion**

Write `need_reply(sentence)`: if the sentence contains "I need X", return "Why do you need X?". For any other sentence, return `None`. Use `normalize`, `re.search` and `group(1)`.

```
>>> print(need_reply("I need a break."))
Why do you need a break?
>>> print(need_reply("Well, I need coffee!"))
Why do you need coffee?
>>> print(need_reply("I needed help yesterday."))
None
```

One function, many rules

`i_am_reply`, `feel_reply` and `need_reply` differ only in the pattern and the start of the reply:

```
def rule_reply(pattern, before, sentence):
    match = re.search(pattern, normalize(sentence))
    if match is None:
        return None
    return before + match.group(1) + "?"

print(rule_reply("^i am (.*)", "Why are you ", "I'm awake."))
print(rule_reply("^i feel (.*)", "Do you often feel ", "I feel cold!"))
print(rule_reply("i need (.*)", "Why do you need ", "Well, I need coffee!"))
```

```
Why are you awake?
Do you often feel cold?
Why do you need coffee?
```

A rule is now two strings: a pattern and a reply start – ELIZA's real design.

Checkout

1. What does `re.search` return when the pattern does not match, and how does `i_am_reply` test for it?

→ `None`. `if match is None: return None` – before `group(1)`, because `None` has no `group` method.

2. Why `r"\bmother\b"` and not `"\bmother\b"`?

→ Without `r`, Python turns `\b` into a backspace before `re` sees it. The raw string keeps the backslash.

Summary

- `re.search(pattern, string)` returns a **match object** or `None`. `group()` is the matched text, `span()` its start and end
- Special characters: `|` or, `^` start, `$` end, `?` the character before is optional, `.` any character, `*` repeat the character before
- Parentheses make a **group**; `group(1)` returns what the first group matched. `"i am (.*)"` cuts out the rest of the sentence
- `re.sub(pattern, new, string)` replaces every match
- `\b` matches a **word boundary**. Write patterns with a backslash as **raw strings**, `r"..."`

Glossary

#	Term	Meaning
1	regular expression	Pattern that describes a set of strings
2	match object	What `re.search` returns when it finds a match
3	special character	Character with a meaning in a pattern: `^` `$` `?`
4	anchor	Pattern part for the start or end of a string
5	group	Pattern part in parentheses, `group(1)`
6	substitution	Replace what a pattern matches: `re.sub`
7	word boundary	Edge between a word and a non-letter: `\b`
8	raw string	`r"..."`: backslashes stay as typed

Code glossary

#	Code	Meaning
1	`import re`	Load the module for regular expressions
2	`re.search(p, s)`	First match of pattern p in s, or None
3	`m.group()`	The text that matched
4	`m.span()`	Start and end index of the match
5	`m.group(1)`	The text that the first group matched
6	`a|b`	a or b
7	`^`	Start of the string
8	`$`	End of the string
9	`?`	The character before it is optional
10	`.`	Any one character
11	`.*`	Any characters, as many as there are
12	`(...)`	A group: the part to cut out
13	`\b`	Word boundary
14	`r"..."`	Raw string: backslashes stay as typed
15	`re.sub(p, new, s)`	Replace every match of p in s by new

Next: lists

`rule_reply` takes one rule at a time. ELIZA needs dozens – and several replies per rule, so it does not repeat itself.

- Next: **lists** – a rule list and a reply list
- and reflection word by word, so "i" no longer hits the "i" in "like"